

课程笔记

LlamaIndex RAG 原理及源码分析

五道口纳什 & Claude

2026 年 3 月 28 日

RAG, 基本原理 llama_index 源码分析 Embed, LLM

视频作者/频道: 五道口纳什

发布日期: 2025

视频时长: 约 22 分钟

视频链接: <https://www.bilibili.com/video/BV1Vm42157PL>

目录

1	RAG 基本概念	2
1.1	RAG 整体框架	2
1.2	两个核心 Model	2
1.3	本章小结	2
2	LlamaIndex 源码分析: Indexing 流程	2
2.1	文档加载与 Chunking	2
2.2	Node 创建与 Embedding	3
2.3	本章小结	3
3	LlamaIndex 源码分析: Query 流程	3
3.1	检索过程	3
3.2	Prompt Template	3
3.3	消息构建与 API 调用	3
3.4	本章小结	4
4	总结与延伸	4

1 RAG 基本概念

1.1 RAG 整体框架

RAG (Retrieval-Augmented Generation) 是一个非常实用、非常基础的现代框架，无论工程还是科研都极为必要。

RAG 核心流程

1. 用户发出一个 **Query** (问题)
2. Query 与外部数据的 **Index** 进行匹配——外部数据指非语言模型之外的、存在硬盘上的数据 (结构化数据库、非结构化文档等)
3. 对外部数据进行 **Chunking** (分块)，每个 Chunk 创建一个 **Embedding Vector**
4. 对 Query 也创建 Embedding Vector
5. 计算 Query 与各 Chunk 之间的**相似度** (点积/余弦)，取 Top-K 最相关的 Chunk
6. 将相关 Chunk 作为 Context，与 Query 一起送入 LLM，生成回答

1.2 两个核心 Model

RAG 系统中需要两个模型：

1. **Embedding Model**：将文本转换为向量表示 (默认 text-embedding-ada-002, 1536 维)
2. **Language Model**：基于 Context 生成回答 (默认 GPT-3.5 Turbo)

RAG 的核心价值

RAG 要求 LLM 仅基于检索出的 Context Information 回答问题，而非使用模型自身已有的知识——降低幻觉，提高对事实性要求高的场景的可靠性。

1.3 本章小结

RAG 的本质是”检索增强生成”：先从外部数据中检索最相关的信息，再让 LLM 基于这些信息回答问题，从而减少幻觉、提高准确性。

2 LlamaIndex 源码分析：Indexing 流程

2.1 文档加载与 Chunking

1. 从 Data 目录加载所有文档 (返回 Document 列表)
2. 对每个文档进行 **Split**——Chunk Size 为 1024 (实际约 997，减去文档路径的 token 数)，Overlap 为 200
3. Split 过程先按自身规则切分 (产生约 759 个 split)，再进行 **Merge** 合并为最终的 Chunk (约 22 个)

2.2 Node 创建与 Embedding

1. 每个 Chunk 创建一个 **Node** (具有唯一 `node_id` 和对应的文本内容)
2. 使用 Embedding Model (text-embedding-ada-002, Batch Size 2048) 为每个 Node 计算 Embedding Vector
3. 最终得到 22 个 Node, 每个 Node 有一个 1536 维的 Embedding Vector (512×3)

2.3 本章小结

Indexing 流程: 文档 $\rightarrow$ Split $\rightarrow$ Merge $\rightarrow$ Chunk $\rightarrow$ Node $\rightarrow$ Embedding Vector。每一步都在源码中有清晰的实现。

3 LlamaIndex 源码分析: Query 流程

3.1 检索过程

1. 对用户 Query 计算 Embedding Vector (同样 1536 维)
2. 计算 Query Embedding 与所有 Node Embedding 的相似度
3. 默认取 **Top-2** 最相关的 Node

相似度计算支持多种度量: 欧式距离、点积、余弦相似度等。

3.2 Prompt Template

QA Template (源码中提取)

核心模板:

Context information is below. {context} Given the context information and not prior knowledge, answer the query. Query: {query}

关键约束: **仅基于 Context Information 回答, 不使用模型已有知识。**

此外还有 Refine Template (基于新 Context 优化已有答案) 和 Summarize Template 等。

3.3 消息构建与 API 调用

最终构建的消息包含:

1. **System Message**: 角色设定——”你是一个值得信赖的 QA 专家, 永远基于 Context Information 回答, 不使用已有知识”
2. **User Message**: 包含 Context Information (Top-K 相关文档) + Query

默认 Response Mode 为 Compact (紧凑型)。

3.4 本章小结

Query 流程: Query Embedding → 相似度匹配 → Top-K Node → Prompt Template 组装 → LLM 生成回答。整个过程只有约五行框架代码, 但内部源码非常精细。

4 总结与延伸

1. **RAG 是一个非常基础且实用的技术**, LlamaIndex 提供了高度封装的实现, 框架层面仅需约五行代码即可完成全流程。
2. **理解源码是掌握框架的关键**: LlamaIndex 封装得非常高级, 但理解内部的 Split/Merge/Node/Embedding/Retrieve/Template 流程才能真正掌握。
3. **核心组件**: Embedding Model 负责向量化, Language Model 负责生成, Prompt Template 负责组织输入, 相似度计算负责检索。
4. **可扩展方向**: 自定义 Embedding Model、调整 Chunk Size 和 Overlap、修改 Response Mode、引入更复杂的 Retrieval 策略等。